ACV-AD

Autonomous Codebase Visualiser & Automated Documenter

Complete Feature Reference — Scope Control Document

This document is the single source of truth for all features in the ACV-AD project. Use it when adding new scope, reviewing priorities, or onboarding contributors. V1 features are in the current build plan. V2 features are scoped for a future release.

01 — CORE PARSING PIPELINE

Multi-Pass AST Ingestor

V1 — CORE

Recursively walks a target Python repository and extracts a complete structural map using Python's native ast module. No regex, no string scraping — purely deterministic.

- Extracts all function definitions: name, arguments, return type annotation, line range
- Extracts all class definitions: name, base classes, method list, line range
- Extracts all top-level imports (plain import and from...import), including aliases
- Extracts call sites inside every function body, including attribute calls (obj.method())
- Extracts module-level global variables
- Handles syntax errors in individual files gracefully — logs error, skips file, continues
- Outputs a nested JSON object keyed by absolute file path

Note: Foundation of the entire product. If this is wrong, everything downstream is wrong.

Cross-File Symbol Resolution Engine

V1 — CORE

Takes raw AST output and resolves all ambiguous call references to fully-qualified canonical identifiers. Eliminates the problem of two files both defining 'execute()'.

- Pass 1: builds a global symbol table mapping every defined symbol to its canonical ID
- Canonical ID format: relative/path/file.py::ClassName.method_name
- Pass 2: resolves each call site by tracing import aliases back to their source file
- Marks unresolvable calls (stdlib, third-party libraries) as external=true
- Handles circular imports without crashing
- Handles relative imports (from . import utils)
- Star imports (from module import *) flagged as ambiguous but not dropped
- Unresolved calls logged separately for transparency

02 — BACKEND API (FASTAPI)

Repository Parse Endpoint

V1 — CORE

POST /api/parse-repo — triggers full parsing of a local directory and caches the result in memory for subsequent requests without re-parsing.

- Accepts absolute directory path
- Validates directory exists and is readable before parsing
- Returns summary stats: node count, edge count, list of files with parse errors
- In-memory cache keyed by directory path — second call returns instantly
- CORS configured for frontend development port (localhost:5173)

Graph Data Endpoint

V1 — CORE

GET /api/graph — returns the full graph payload (nodes + edges) for the currently loaded repository, translated into frontend-ready types.

- Returns GraphNode list with id, kind, label, file, line_start, line_end
- Returns GraphEdge list with source, target, kind (calls/imports/defines)
- Node kinds: directory, file, class, function, variable
- Edges are directed — call direction is preserved

Function Source Endpoint

V1 — CORE

GET /api/function-source — returns the raw source lines of a specific function by its canonical node ID.

- Accepts canonical node_id as query parameter
- Returns source code string, file path, and exact line range
- Returns 404 with clear message if node_id not found

03 — CODEBASE GRAPH VISUALISATION

Interactive Codebase Graph Canvas

V1 — CORE

The centrepiece of the product. An interactive directed graph showing all files, classes, and functions in the repository and how they call each other. Built with React Flow on a dark, developer-focused aesthetic.

- Directed edges with arrowheads showing call direction
- Custom node shapes and colours per kind: directories (blue), files (green), classes (purple), functions (orange)
- Automatic initial layout using dagre (top-to-bottom directed acyclic graph)
- Full drag, zoom, and pan — hardware accelerated
- MiniMap in bottom-right corner for large graph navigation
- Zoom controls panel (zoom in / zoom out / fit view)
- Dot-grid background in dark theme
- Click node to select — triggers sidebar highlight and right pane update
- Double-click node to zoom and fit to that node's immediate neighbourhood
- Right-click context menu: Document / Impact Analysis / View Source
- Warning banner when graph exceeds 300 nodes

Note: Uses React Flow (@xyflow/react), not Pyvis. This is non-negotiable for quality.

Graph Node Context Menu

V1 — CORE

A floating context menu that appears on right-click of any graph node, providing quick access to the three main actions.

- Document — opens AI documentation pane for the selected node
- Impact Analysis — runs upstream caller analysis for the selected node
- View Source — displays the raw function source code inline
- Closes on outside click or Escape key

04 — SIDEBAR NAVIGATION

Repository Loader

V1 — CORE

The top section of the sidebar. Allows users to enter a local repository path and trigger parsing with real-time feedback.

- Absolute path text input with submit button
- Spinner shown during parsing
- On success: displays node count, edge count in green
- Parse errors shown in collapsible warning section with affected filenames
- Last used path persisted in localStorage across sessions

File Tree Browser

V1 — CORE

A collapsible tree of the parsed repository structure, grouped by directory → file → function.

- Three-level hierarchy: directory / file / function
- Function count badge displayed per file
- Click a function: selects and highlights that node in the graph
- Click a file: filters graph to show only that file's nodes and direct connections
- Active selection highlighted in blue
- Directories with more than 5 children collapsed by default

Real-Time Filter Bar

V1 — CORE

A search input above the file tree that filters the tree in real-time as the user types.

- Matches on both file names and function names simultaneously
- Clears with Escape key or × button
- Shows live match count: '12 matches'

View Mode Toggle

V1 — CORE

Switches the main canvas between Codebase view (cross-file call graph) and File view (single-function execution flowchart).

- Codebase mode: full cross-file dependency and call graph
- File mode: select a specific function, render its internal execution flowchart
- Rendered as a segmented toggle button in the sidebar

05 — IMPACT ANALYSIS

Upstream Impact Analysis Engine

V1 — CORE

Given a selected function, answers: 'If I change this, what breaks?' Traverses the call graph in reverse to find all direct and transitive callers, showing the full blast radius of any change.

- Reverse index built at init for O(1) upstream lookups — not recomputed per request
- BFS traversal upstream from target node
- Separates direct callers (one hop) from transitive callers (multi-hop)
- Configurable max_depth (default: 5) to prevent infinite traversal
- Detects and reports circular call chains without crashing
- Returns full ImpactResult: target node, direct callers, transitive callers, all edges

Note: This is the feature that makes the tool genuinely useful for refactoring decisions.

Impact Analysis Pane

V1 — CORE

The right-side panel that displays impact analysis results for a selected function.

- Shows direct callers section with file and function name
- Shows transitive callers section with call chain path displayed
- Circular dependency warning displayed when detected
- Empty state message when function has no callers (entry point)
- Clicking any caller in the list selects that node in the graph
- 'Highlight in Graph' button animates all blast radius edges in the canvas

06 — AI DOCUMENTATION

Context-Isolated Documentation Pipeline

V1 — CORE

When 'Document' is selected on a function, generates structured markdown documentation using an LLM. Context is deliberately constrained to avoid token bloat.

- Context includes: target function source, direct callee signatures, direct caller signatures, parent class definition (if a method)
- Context ceiling: approximately 2,000 tokens regardless of repository size
- Structured output format: Purpose, Parameters, Returns, Side Effects, Notes
- Streaming response — text appears progressively, not all at once
- Developer-persona system prompt — no basic Python explanations, senior-engineer tone

Note: Constrained context is a deliberate architectural decision. Do not change it without reason.

Multi-Provider AI Support

V1 — CORE

Supports local and cloud AI providers via LiteLLM — one SDK for all providers.

- Ollama (local): llama3 via http://localhost:11434 — no API key required
- OpenAI: gpt-4o-mini via OPENAI_API_KEY environment variable
- Anthropic: claude-haiku-4-5 via ANTHROPIC_API_KEY environment variable
- Provider toggle in the documentation pane UI
- Graceful error message if selected provider is unavailable

AI Documentation Pane

V1 — CORE

The right-side panel that renders streamed AI documentation for a selected function.

- Markdown rendered with syntax highlighting (react-markdown + rehype-highlight)
- Text streams in progressively with blinking cursor
- Provider selector: Ollama / OpenAI / Anthropic toggle
- Save Documentation button — persists to .visualiser/ directory
- Regenerate button — re-runs documentation with same or different provider
- Empty state: 'Select a function node and click Document'
- Error state: clear message with fix instructions (e.g. 'Ollama not running')

07 — FILE-LEVEL EXECUTION FLOWCHART

Function Execution Flowchart

V1 — CORE

When in File view mode, renders the internal execution logic of a selected function as a standard flowchart using conventional notation. Shows HOW a function works, not who calls it.

- Oval / rounded rectangle: function entry point and return statements
- Rectangle: assignments, expressions, plain function calls
- Diamond: if / elif / else / match decision branches
- Parallelogram (skewed rectangle): print, input, file I/O operations
- Double-bordered rectangle: calls to other defined functions (sub-processes)
- Decision edges labelled 'Yes' / 'No' for conditional branches
- Loop constructs (for/while) rendered with back-edge to decision node
- Top-to-bottom layout via dagre
- Hover any node to see source line number tooltip
- Same dark theme as codebase graph

08 — LOCAL PERSISTENCE

Flat-File Storage Layer

V1 — CORE

Saves graph state and documentation locally inside the inspected repository under a .visualiser/ hidden directory. Zero database setup required. Files commit to git naturally alongside source code.

- .visualiser/ directory created automatically on first save
- graph_state.json: saves node x/y positions so layout is preserved across sessions
- metadata.json: parse stats and last-updated timestamp
- docs/{hash}.md: one markdown file per documented function
- docs/index.json: maps node canonical ID to documentation filename
- Auto-save node positions every 60 seconds if nodes have been moved
- On repo load: restores saved layout positions and previously generated documentation

Note: Files are plain text — readable and diffable without the tool.

09 — PACKAGING & OPEN-SOURCE DISTRIBUTION

Docker Compose Single-Command Setup

V1 — CORE

The entire stack starts with one command: docker compose up. Critical for open-source adoption — contributors must not spend more than 2 minutes getting a working environment.

- Backend (FastAPI) and frontend (React/Vite) as separate services
- Frontend accessible at http://localhost:5173
- Backend API at http://localhost:8000
- Repository path mounted as a read-only volume into the backend container
- AI provider API keys passed via environment variables (not hardcoded)
- .env.example documents all configurable variables
- .env in .gitignore — no secrets committed

README — Product-First Documentation

V1 — CORE

The README is the product's front door. Written as a product page, not a technical manual.

- Hero section: one-sentence value proposition
- UI screenshot prominently placed above the fold
- Quick Start section: 3 commands to a running tool
- Local AI section: how to run with Ollama (zero API key)
- Stack summary
- Contributing guide

10 — V2 FUTURE SCOPE (NOT IN CURRENT BUILD)

Function Performance Prediction

V2 — FUTURE

Estimate the computational complexity and relative performance cost of individual functions and their call chains, surfaced directly on the graph.

- Cyclomatic complexity calculated from AST (number of decision branches)
- Call chain depth score: how many nested function calls a function triggers
- Loop detection: flags functions containing nested loops as high-complexity
- Complexity score overlaid on graph nodes as a colour heatmap (green → red)
- Performance report pane: ranked list of highest-complexity functions
- Drill-down: select a function to see which sub-calls contribute most to its score

Note: Foundation already exists in TASK-02 ResolvedGraph. Add complexity metrics on top.

Multi-Language Support

V2 — FUTURE

Extend the AST parsing pipeline to support languages beyond Python.

- JavaScript/TypeScript via Babel AST or tree-sitter
- Java via tree-sitter-java
- Go via go/ast package (requires Go subprocess)
- Language auto-detection from file extensions
- Unified GraphPayload schema regardless of source language

Note: Requires a language-agnostic parser abstraction layer. Design this before implementing.

Team Collaboration Mode

V2 — FUTURE

Allow multiple engineers to view and annotate the same codebase graph simultaneously.

- Shared documentation: push .visualiser/ docs to a central store (S3 or git)
- Annotation layer: engineers can add notes to any node without generating AI docs
- Change tracking: diff graph state between two git commits to show what changed
- Slack/Teams integration: share a node's documentation as a message

Note: Requires a backend storage upgrade from flat files to a real persistence layer.

VS Code Extension

V2 — FUTURE

Embed the graph canvas directly inside VS Code as a panel, so engineers never leave their editor.

- Webview panel using VS Code extension API
- Opens graph centred on the currently open file
- Click a node in the graph to jump to that function in the editor
- Right-click a function in the editor to trigger impact analysis

11 — BUILD ORDER SUMMARY

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '**Task**' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Task', textColor=Color(.901961,.929412,.952941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '**Feature**' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Feature', textColor=Color(.901961,.929412,.952941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '**Version**' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Version', textColor=Color(.901961,.929412,.952941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
TASK-01	Multi-Pass AST Ingestor	V1
TASK-02	Cross-File Symbol Resolution Engine	V1
TASK-03	FastAPI Backend & API Layer	V1
TASK-04	React Frontend Scaffold	V1
TASK-05	Interactive Codebase Graph Canvas	V1
TASK-06	Sidebar Navigation (Loader + Tree + Filter)	V1
TASK-07	Impact Analysis Engine + Pane	V1
TASK-08	AI Documentation Pipeline + Pane	V1
TASK-09	File-Level Execution Flowchart	V1
TASK-10	Flat-File Persistence Layer	V1
TASK-11	Docker Compose + README	V1
—	Performance Prediction	V2
—	Multi-Language Support	V2
—	Team Collaboration Mode	V2
—	VS Code Extension	V2